TP1 Multithreading

Exercice 1: Exécution séquentiel vs Concurrente

1. Ecrire un programme qui permet d'exécuter deux traitements séquentiels :

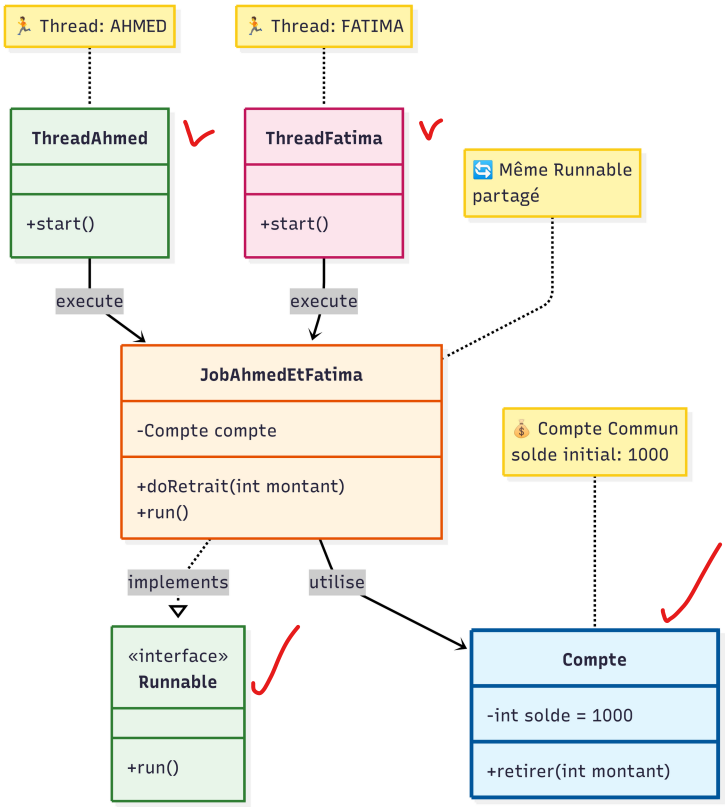
- T1 : une boucle qui permet d'afficher les nombres de 1 à 10
- T2 : une boucle qui permet d'afficher les lettres de « a » jusqu'à « z »

2. Créer deux threads qui permettent de réaliser les deux traitements de façon concurrente.
3. Peut-on déterminer le résultat d'exécution (dans les deux cas) ? Que peut-ont conclure ? dans le premier cas on peut anticiper le resultat mais dans le cas concurential des thread on peut pas .
4. Réécrire les classes en utilisant l'interface Runnable
5. Quelle est la différence entre l'utilisation de la classe Thread et l'interface Runnable. Utiliser Thread (extends Thread) mélange la tâche et son exécution et empêche d'hériter d'une autre classe, tandis que Runnable (implements Runnable) définit seulement la tâche, plus flexible et réutilisable, pouvant être exécutée par un Thread ou un ExecutorService tout en permettant l'héritage.
6. Modifier les priorités des threads : lettres à MAX_PRIORITY et chiffres à MIN_PRIORITY. Qu'est ce que vous remarquez ?
7. Modifier le main afin que l'affichage des chiffres se terminer avant de continuer l'affichage des lettres (utiliser la méthode join())

Exercice Accès concurrent

Le but de cette exercice est de simuler des opérations sur un compte bancaire Commun

- Une classe Compte comprenant
  - Un attribut privé solde initialisé à 1000 (solde courant du compte)
  - Une méthode retirer(int montant) permettant de retirer un certain montant du compte.
- La classe JobAhmedEtFatima implémente Runnable et représente le comportement que Ahmed et Fatima ont tous les deux. Cette classe contient :
  - Un attribut compte représentant le compte en banque commun de Ahmed et Fatima.



- Une méthode doRetrait(int montant) permettant à Ahmed ou Fatima de réaliser un retrait sur leur compte en banque.
  - Vérifier le solde, puis s'endort 200 ms, puis à son réveil (au bout des 200 ms) réalise le retrait.
  - Le nom de la personne réalisant le retrait doit être signalé.

- Une méthode run() décrivant le comportement de Ahmed et Fatima:

effectuer en boucle des retrait de 10.

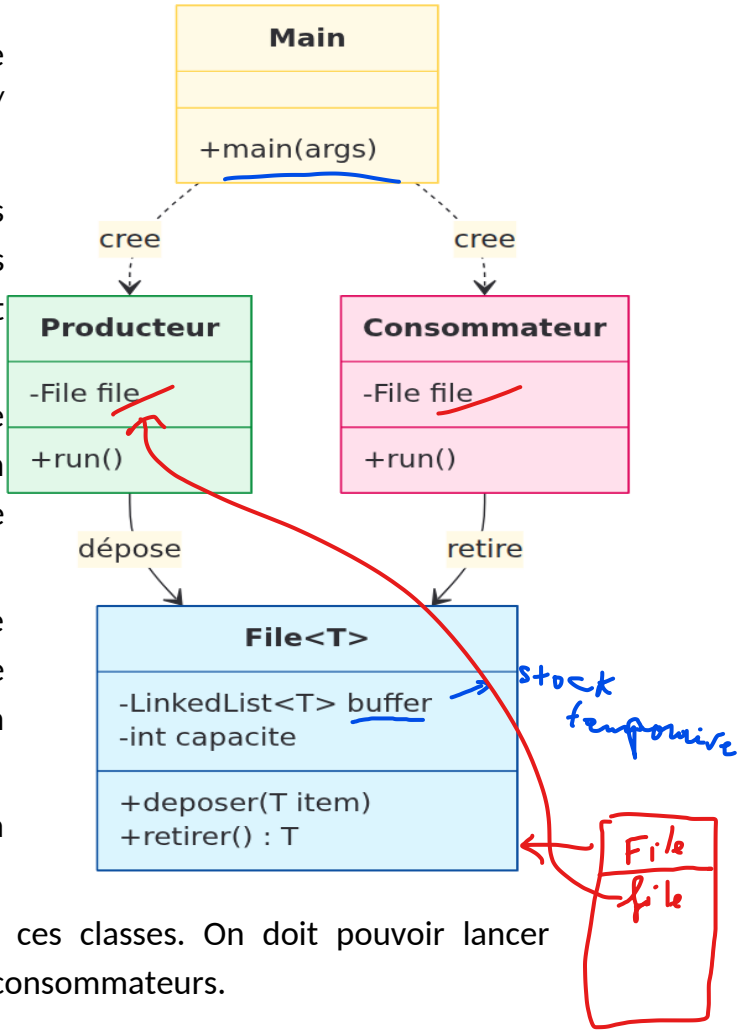
- La méthode main(String[] args) crée deux threads (Ahmed et Fatima) avec le même Runnable, les nomme, puis les lance.

1. Pourquoi utiliser un runnable ? pour isoler le thread de sa tache et utiliser le runnable pour les deux threads
2. Qu'est ce que vous remarquez lors de l'exécution? le simple withrawal est le plus missorganised et ses resultats ne son pas coherents
3. Pourquoi le sleep est crucial ? Pour garantir la securiter et d'eviter le fait que les deux uttilsateurs fait un retrait au meme temps et compter juste une fois et donner une erreure
4. Comment résoudre le problème ?
5. Utiliser un lock/ReentrantLock
6. Quelle est la différence l'usage de Lock et Synchronized ? ils ont les memes objectifs mais le lock donne plus de fonctions comme tryLock qui donne un verrou au thread suivant meme si il n est pas fini et de plus de gestion des interuptions

Exercice 3 Producteur/consommateur

Ecrire un programme java qui permet de simuler le problème producteurs/ consommateurs.

- Les threads producteurs créent des produits (par exemple un entier) et les placent dans une file d'attente (LinkedList comme FIFO).
- Chaque thread consommateur retire de la file d'attente un item produit par un producteur et le consomme (par exemple l'affiche).
- La file d'attente est bornée: lorsque la file d'attente est pleine un producteur se met en attente. Lorsqu'elle est vide un consommateur se met en attente.
- On devra définir trois classes: File pour la file, Producteur et Consommateur



- Une classe Main permettant de tester ces classes. On doit pouvoir lancer simultanément plusieurs producteurs et consommateurs.

Exercice 6 Travail en parallèle

On souhaite écrire un programme qui affiche les valeurs des racines carrées de 0 à 5000. On souhaite paralléliser le programme et permettre d'exécuter en même temps plusieurs calculs. Chaque thread devra par exemple effectuer le calcul d'une centaine de racine carré.

1. Ecrire la version séquentielle de ce travail. Mesurer le temps d'exécution nécessaire (System.currentTimeMillis();)
2. Créer un ExecutorService en utilisant la classe Executors permettant de distribuer le calcul sur 5 threads différentes.
3. Écrire le programme qui effectue le calcul en parallèle en utilisant des Runnable qui effectuent l'affichage des résultats.
    - On permet que l'affichage ne s'effectue pas dans l'ordre des racines carrés croissantes.
4. Utiliser la méthode shutdown pour indiquer qu'il n'y a plus de calcul à effectuer.
5. Modifier votre programme en utilisant l'interface Callable (classe séparée RootTask) à la place de Runnable et le mécanisme de Future pour effectuer l'affichage dans l'ordre.
    - Utiliser un collection pour stocker les objets future
    - Effectuer un parcours sur la collection pour récupérer les résultats
6. Mesurer le temps d'exécution nécessaire pour la version Callable

0 - Multithreading simple → extends Threads
1 - Runnable → implement Runnable
2 - NameThread.setPriority(Thread.Max_Priority)
3 - NameThread.join();
4 - Synchronized -
5 - Lock / ReentrantLock
6 - Withdrawl (pour entrer les méthodes comme arguments dans d'autre méthode)
7 - Executors - → Runnable
→ callable